
Practice Exam: Deep Learning and Deployment

2026-06-06

Question 1. (1 points)

What is the fundamental difference in how 'CMD' and 'ENTRYPOINT' are used in a Dockerfile when launching a container?

- A) 'CMD' sets a default command that can be easily overridden by providing arguments to 'docker run', while 'ENTRYPOINT' configures the container to run as a specific executable.
- B) 'ENTRYPOINT' is used to install software packages with 'apt-get' or 'pip', while 'CMD' is used to run the application after installation.
- C) A Dockerfile can have multiple 'CMD' instructions to run a sequence of commands, but only one 'ENTRYPOINT' to specify the final step.
- D) 'CMD' is an older, deprecated instruction, and modern Dockerfiles should always use 'ENTRYPOINT' for defining the container's main process.

Question 2. (1 points)

When would packaging your Python project as a pre-compiled wheel (.whl) file be most advantageous compared to distributing the source code for installation with pip?

- A) When the project is pure Python with no external dependencies, as this is the only scenario where wheels can be created.
- B) When distributing a private package within an organization, as it allows for faster and more reliable installations by avoiding a build step on the target machine.
- C) When the project's code needs to be easily auditable, as the wheel format is a transparent, uncompressed archive.
- D) When you need to include large data files, such as trained model weights, directly within the Python package.

Question 3. (1 points)

The Universal Approximation Theorem is a cornerstone of deep learning. Which of the following is an **INCORRECT** interpretation of what the theorem implies?

- A) A shallow neural network with two linear layers and activations can, in principle, approximate any continuous function if it has enough hidden units.
- B) The theorem guarantees that a neural network can learn any function.
- C) The theorem proves that a set of optimal parameters exists for the network to approximate the function, but it does not provide a method for finding them.
- D) The theorem implies that a sufficiently wide network will always find the best possible parameters through gradient descent.

Question 4. (1 points)

Which of the following statements about tensors is **NOT** correct?

- A) A tensor with 3 dimensions (eg (batch, sequence, embedding)) can be represented as a cube of numbers.
- B) Broadcasting in PyTorch allows tensors to be combined when dimensions are either equal or one of them is 1. Eg when adding a tensor A of shape $(1, 3)$ to another tensor B of shape $(32, 3)$, the result will be the same as when tensor A would have been concatenated 32 times to create a $(32, 3)$ tensor.
- C) A CNN can process 3D tensors directly, e.g. pictures of trees you took with a black-and-white camera that you stored as 3D-tensors (batch, width, height), without any modification
- D) 5D tensor are used to represent a batch of colored video frames

Question 5. (1 points)

When training a neural network, what **best** describes what the mechanism of gradient descent learning does?

- A) Providing a measure of the error between prediction ($\hat{y}$) and target (y)
- B) Providing the direction in which the weights should be updated
- C) Updating the weights in the network to minimize loss by moving in the direction of the negative gradient.
- D) Determining the optimal learning rate dynamically to optimize the loss

Question 6. (1 points)

In batch normalization, there is an epsilon (ϵ) term, where ϵ prevents division by zero when the variance (σ^2) approaches zero. Which activation function would you expect to benefit the most from this ϵ term?

- A) Sigmoid
- B) ELU
- C) Leaky ReLU
- D) ReLU

Question 7. (1 points)

What was the **key architectural innovation** of the GoogleNet Inception model?

- A) It introduced residual connections to enable the training of much deeper networks by mitigating vanishing gradients.
- B) It used parallel modules with multiple kernel sizes (e.g., 1×1 , 3×3 , 5×5) and a pooling operation, allowing the network to capture features at various scales simultaneously.
- C) It was the first model to heavily rely on very small (3×3) convolutional filters stacked on top of each other to create a deep and simple architecture.
- D) It simplified network design by removing all pooling layers and relying solely on convolutions with an increasing stride to reduce spatial dimensions.

Question 8. (1 points)

Consider a 4D input tensor representing a batch of 16 satellite images, with dimensions $(16, 8, 28, 28)$ in $B \times C \times H \times W$ format. The tensor is passed through a 2D convolutional layer with 32 filters, a kernel size of 3×3 , a stride of 1, and no padding. What are the dimensions of the output tensor?

-
- A) (16, 32, 26, 26)
 - B) (16, 32, 28, 28)
 - C) (16, 32, 14, 14)
 - D) it will crash without aggregation: images should have 1 or 3 channels, not 8

Question 9. (1 points)

What is the **core** mathematical reason that simple RNNs are susceptible to the vanishing gradient problem when processing long sequences?

- A) They use activation functions like tanh or sigmoid which saturate, causing their derivatives to become zero for large inputs.
- B) The gradient signal from must be backpropagated over the full sequence length.
- C) They can only remember information for a fixed number of time steps, determined by the size of the context window.
- D) They lack a dedicated "cell state" for storing long-term memories separately from the short-term hidden state.

Question 10. (1 points)

Inside an GRU cell at a specific timestep t , the new hidden state h_t is calculated with $h_t = \Gamma_u \otimes h_{t-1} + (1 - \Gamma_u) \otimes \tilde{h}_t$. If the gate Γ_u has values of 0, what is the **immediate effect**?

- A) The influence of the previous cell state h_{t-1} is almost completely erased, and the new cell state h_t is formed almost exclusively from the current input.
- B) The current input, represented by the candidate cell state $\tilde{h}_t$, is blocked from influencing the new cell state h_t .
- C) This is a forget state, and thus the cell state h_t is reset to a zero vector.
- D) Without skiplayers the learning process for the network halts, as the gradients are prevented from flowing backward through the cell state.

Question 11. (1 points)

Consider a timeseries $X = \{x_0, x_1, \dots, x_n \mid \forall x \in \mathbb{R}\}$ where x_i or x_j is the i -th (or j -th with $i \neq j$) timestep in a timeseries of n steps and an kernel of length $n > k > 1$. Assume the direction of the time dimension goes from 0 to n . If we allow for variable-length, this means that two timeseries X and X' have different values for n . Which of the following **accurately describes** how 1D convolutions (`torch.nn.Conv1d`) work for time series analysis?

- A) 1D convolutions scan through the time dimension with a sliding kernel, detecting local patterns across multiple time steps
- B) 1D convolutions process each time step i independently of every other timestep j , reusing the kernel for efficiency and applying the same transformation to each point in the sequence,
- C) 1D convolutions require a fixed length, therefore you will always will need padding for variable-length timeseries, for every batchsize
- D) 1D convolutions use a gate to learn when to remember and when to forget

Question 12. (1 points)

What is the **advantage** of using Random Search over Grid Search for hyperparameter tuning, especially when dealing with a **big high-dimensional search space** that has a total of n possible combinations? Assume you have a budget of k experiments, with $k \ll n$ meaning k is small compared to n .

-
- A) Random Search makes sure the model can not over-specialize and thus protects against overfitting.
 - B) Random Search explores random parts of the hyperparameter space, while with Grid Search you need to define the grid explicitly which probably suffers from human bias.
 - C) Randomization improves the explore-exploit balance compared to Grid Search; this helps against getting stuck in a local minima
 - D) High dimensional spaces have unexpected behavior. Randomness guards us against this ‘curse of dimensionality’

Question 13. (1 points)

What is the **primary** motivation for using a learning rate ”warm-up” schedule at the very beginning of the training process for large, deep neural networks?

- A) To allow the model to quickly converge to a good region of the loss landscape.
- B) To prevent the optimizer from taking destructively large steps while the model’s weights are still randomly initialized, overfitting on the first few batches.
- C) To make the model train faster by using a very high learning rate for the first few epochs.
- D) To automatically find the best possible initial learning rate before the main training schedule begins.

Question 14. (1 points)

You decide to tune a deep learning model with the following hyperparameter space:

- Hidden units per layer: between 1-512 (2^8)
- Number of layers: between 1-8 (2^3)
- Optimizers: [Adam, SGD, RMSprop, AdamW] (2^2)
- Activations: [ReLU, SeLU, GeLU, LeakyRelu] (2^2)
- Dropout rate: [0.1, 0.2, 0.3, 0.4] (2^2)
- Kernel sizes: [3, 5, 7, 9] (2^2)
- Number of filters: between 1-256 (2^8)

This comes down to a searchspace of 2^{27} experiments. On average, you can train 16 experiments per hour and your team has access to a cluster of 8 VMs, each with 64 cores, which you are willing to train for a full 4 hours. You have been bragging all week about the impressive 2^{15} experimental budget this gives you. Which hyperparameter tuning strategy with Ray is expected to give the **best final results**? See the addendum for some powers of two.

- A) Use grid search with Ray to systematically explore all possible combinations. You will still cover a pretty big chunk of the searchspace, which will be more than enough.
- B) Due to the massive cluster, you covered the exploit and should use Random Search with Ray to maximize your explore-exploit balance.
- C) Use Ray with Hyperband to aggressively prune poorly performing trials early, combined with a Tree Parzen Estimator
- D) Because of you massive amount of VMs, brute force is an option; you dont need aggressive pruning; so simply use a Tree Parzen Estimator. Exiting trials early will make it harder to compare between runs, making it less effective.

Question 15. (1 points)

In a medical diagnosis model for detecting a rare disease (prevalence is low), the confusion matrix shows: TP=4, FP=1, TN=979, FN=16. Recall is $\frac{TP}{TP+FN}$ and precision is $\frac{TP}{TP+FP}$. Based on this information, what ethical trade-off issue is **most apparent**?

- A) The model prioritizes precision over recall, missing many cases of the disease
- B) The model prioritizes recall over precision, leading to false alarms
- C) The model is unbiased and represents an optimal balance between false positives and false negatives.
- D) The dataset is too small for meaningful analysis of the performance on a rare disease.

Question 16. (1 points)

What is the **fundamental characteristic** of the self-attention mechanism that **distinguishes** its approach to modeling relationships in a sequence from both RNNs and 1D CNNs?

- A) It specializes in modeling long-range dependencies by design, making it more powerful than CNNs for distant relationships but inherently less effective for local patterns.
- B) It directly computes an interaction score between any two positions in the sequence, making its ability to model relationships independent of the distance between tokens.
- C) It processes the sequence step-by-step, passing a hidden state that accumulates information, which is most effective for capturing the influence of the immediately preceding token.
- D) It applies a shared, fixed-size kernel across the sequence, making it highly efficient at detecting recurring local patterns but unable to directly model non-local dependencies in a single step.

Question 17. (1 points)

In the classic transformer architecture, how does the self-attention mechanism ($\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$) operate to generate a representation for a **single** token within this context window?

- A) It turns a single token's representation into a hologram—a weighted sum encoding information from the entire sequence, viewed from that token's perspective
- B) It sequentially processes the context, compressing the information into a fixed-size state vector that is updated at each step, ensuring a linear increase in computational cost.
- C) It partitions the large context window into smaller, fixed-size chunks and processes each chunk independently to maintain computational feasibility, only combining the results at the final layer.
- D) It primarily attends to a small, local window of tokens around the target token, using positional encodings to retrieve general information from more distant parts of the context.

Question 18. (1 points)

In which of the following scenarios would a Siamese Network be the **most appropriate** architectural choice over a standard multi-class classification network (like ResNet)?

- A) Building a system to classify medical scans into 5 disease categories.
- B) A signature verification system for a bank, which must be able to verify new clients' signatures against their records.
- C) A model to predict the age of a person from a photograph in their passport
- D) An unsupervised system to compress high-resolution satellite images into low-dimensional feature vectors for efficient storage.

Question 19. (1 points)

What is the **key difference** in the training process of a Denoising Autoencoder compared to a standard Autoencoder?

- A) The Denoising Autoencoder's loss function is calculated between the noisy input and the reconstructed output.
- B) The input fed into the Denoising Autoencoder's encoder is an artificially corrupted version of a data sample, while the model's output is compared against the original, clean data sample to calculate the loss.
- C) A Denoising Autoencoder requires an orders of magnitude larger bottleneck (the dimensionality of the latent space) to retain enough information to remove the noise.
- D) Denoising Autoencoders are typically used with text data, whereas standard autoencoders are used for images.

Question 20. (1 points)

What is the **primary function** of the latent space in an autoencoder architecture for anomaly detection, represented mathematically as a mapping $enc: \mathbb{R}^d \rightarrow \mathbb{R}^{d_m}$ for the encoder?

- A) To minimize reconstruction loss on all possible input data, both normal and anomalous
- B) To capture a compressed representation that effectively models normal patterns while abstracting away irrelevant variations
- C) To transform input data into a higher-dimensional space where $d_m > d$ for better discrimination
- D) To place all inputs into clusters in the latent space, with anomalies forming distinct clusters from normal examples

Question 21. (1 points)

You are tasked with using spectral clustering to partition the nodes of the (connected) Zachary's Karate Club graph into two distinct communities. You have already computed the graph Laplacian matrix and its eigenvalues and eigenvectors. Which of the following provides the necessary information to create the primary partition?

- A) The eigenvector corresponding to the second-smallest eigenvalue.
- B) The eigenvector corresponding to the smallest eigenvalue.
- C) The eigenvector corresponding to the largest eigenvalue of the Laplacian, which is the most important eigenvalue.
- D) The diagonal entries of the Laplacian matrix, which represent the degree of each node.

Question 22. (1 points)

Which of the following **best** describes why hyperbolic embeddings are useful for certain types of data, as opposed to euclidian embeddings?

- A) They require less computational power than Euclidean embeddings, because it is easier to be "close" in a hyperbolic space
- B) They can efficiently represent grid-like structures with equal distances between all points due to how they handle distance
- C) They are particularly effective for embedding data with hierarchical structure or tree-like relationships, because there is increasingly more space going further from the origin

-
- D) They always produce lower dimensional representations than equivalent Euclidean embeddings, because distance is handled differently and thus more items can be packed in the same area

Question 23. (1 points)

How does a single layer in a Graph Convolutional Network (GCN) update the feature representation for a specific target node?

- A) It computes the new representation by aggregating the features of the target node itself and its immediate neighbors.
- B) It applies a standard 2D convolutional filter to the node's local neighborhood, treating it as a small image.
- C) It calculates the shortest path from the target node to all other nodes and uses the path lengths to update its features.
- D) It uses a self-attention mechanism to compute the importance of every other node in the graph, regardless of distance, to update the target node's features.

Question 24. (1 points)

Which of the following statements is **CORRECT**?

- A) In an MDP, the transition function $T(s, a, s')$ is deterministic, whereas in a POMDP, the transition function is probabilistic.
- B) In an MDP, the agent knows the exact current state s , whereas in a POMDP, the agent relies on observations o to maintain a belief state $b(s)$ representing the probability distribution over possible states.
- C) POMDPs require the state space to be continuous, while MDPs are restricted to discrete state spaces.
- D) An MDP cannot be solved using Reinforcement Learning, whereas a POMDP requires Reinforcement Learning methods to find an optimal policy.

Question 25. (1 points)

Swarm intelligence algorithms (such as Grey Wolf Optimization or Ant Colony Optimization) have major upsides for some problems when compared to gradient-based methods (like Newton-Raphson or Gradient Descent). But what is a **major downside** of using swarm-based approaches?

- A) They cannot be applied to "black-box" optimization problems where the derivative of the objective function is unknown.
- B) They are guaranteed to converge to the global optimum, but they require significantly more memory than gradient-based methods.
- C) They generally lack strong mathematical guarantees for convergence to the global optimum and can be computationally expensive due to the need to evaluate the fitness of a population.
- D) They are unable to handle multimodal landscapes (functions with multiple local peaks).

Question 26. (1 points)

In the context of Markov Decision Processes (MDPs), what does the **optimal policy** $\pi^*(s)$ represent?

- A) A probability distribution over all possible states that maximizes the expected reward in the next step only.

-
- B) A mapping from each state s to an action a that maximizes the expected cumulative discounted reward over time.
 - C) A fixed sequence of actions that must be executed regardless of the current state.
 - D) A function that maps state-action pairs to transition probabilities, defining how the environment responds to actions.

1 Addendum: powers of two

$2^1 = 2$	$2^5 = 32$	$2^9 = 512$	$2^{13} = 8192$
$2^2 = 4$	$2^6 = 64$	$2^{10} = 1024$	$2^{14} = 16384$
$2^3 = 8$	$2^7 = 128$	$2^{11} = 2048$	$2^{15} = 32768$
$2^4 = 16$	$2^8 = 256$	$2^{12} = 4096$	$2^{16} = 65536$

Answer Key

1. deployment-001 — correct: A

Learning goals: 9.5: The student understands the difference between CMD and ENTRYPOINT

- A 'ENTRYPOINT' defines the main executable, and 'CMD' provides default arguments to that executable. Arguments passed to 'docker run' will override the 'CMD' value but will be appended to the 'ENTRYPOINT' executable.
- B Both 'apt-get' and 'pip' installations should be performed using the 'RUN' instruction during the image build process.
- C Only the last 'CMD' and the last 'ENTRYPOINT' instruction in a Dockerfile will have an effect.
- D Both 'CMD' and 'ENTRYPOINT' are current, valid instructions with distinct use cases. Using them together is a common and powerful pattern.

2. deployment-002 — correct: B

Learning goals: 10.6: what a '.whl' file is and why/how it is used

- A Wheels are particularly useful for python packages with compiled C/Rust backends, as they ship the pre-compiled binary, saving the end-user from needing a compatible compiler toolchain.
- B The primary benefit of a wheel is that it's a pre-built distribution - installation from a wheel is faster and more reliable because it's just a matter of unpacking files, bypassing the potentially complex and failure-prone 'setup.py build' step.
- C While a wheel is a ZIP archive, it contains compiled binaries which are not easily auditable - a source distribution ('sdist') is much better for auditing.
- D While possible, including large data files in a wheel is generally considered bad practice - such assets are typically downloaded separately or managed via other means.

3. les1-foundations-001 — correct: D

Learning goals: 1.3: Hoe neurale netwerken een universal function zijn.

- A This is a correct interpretation of the theorem, emphasizing 'in principle' and 'enough hidden units'.
- B This is a correct interpretation highlighting that non-linearity is crucial for universal approximation.
- C This is a correct interpretation noting that UAT is an existence theorem without guaranteeing how to find the solution.
- D This is incorrect because the theorem makes no guarantees about the training process or finding optimal parameters through gradient descent.

4. les1-foundations-002 — correct: C

Learning goals: 1.1: Wat tensors zijn en voorbeelden voor de dimensies tm 5D

- A This statement is correct about 3D tensor visualization.
- B This is a correct explanation of broadcasting in PyTorch.
- C This is incorrect because a CNN can only process 4D tensors as input.
- D This is a correct use case for 5D tensors.

5. les1-foundations-003 — correct: C

Learning goals: 1.4: Wat gradient descent doet en wat een gradient is

- A This describes the loss function, not gradient descent.
- B This describes the gradient itself, not the complete gradient descent process.
- C This is correct as it describes the complete optimization process of gradient descent.

D This confuses gradient descent with adaptive learning rate methods or schedulers.

6. **les2-cnns-001** — correct: **D**

Learning goals: 1.8: Wat een Activation function is, en kent er een aantal (ReLU, Leaky ReLU, ELU, GELU, Sigmoid), 2.8: Begrijpt wat overfitting is en hoe regularisatie (batchnorm, splits, dropout, learning rate) helpt.

- A Sigmoid has natural bounds and doesn't typically produce zero variance.
- B ELU has a negative component which helps prevent zero variance.
- C Leaky ReLU has a small slope for negative values, reducing likelihood of zero variance.
- D** ReLU is correct as it can create exact zeros, leading to zero variance situations where epsilon is crucial.

7. **les2-cnns-002** — correct: **B**

Learning goals: 2.7: Kent ruwweg de innovaties die de verschillende architecturen doen: AlexNet, VGG, GoogleNet, ResNet, SENet

- A This describes the innovation of ResNet, not GoogleNet.
- B** The core idea of the Inception module was to move away from simply stacking layers deeper and instead make the network wider, with parallel paths for feature extraction at different scales within the same module.
- C This is the defining characteristic of the VGG family of networks.
- D While modern architectures sometimes replace pooling with strided convolutions, this was not the innovation introduced by GoogleNet (Inception v1), which explicitly used max pooling within its modules.

8. **les2-cnns-003** — correct: **A**

Learning goals: 2.5: Wat een stride en padding zijn., 5.9: kent het verschil in dimensionaliteit van tensors (2D tensors, 3D tensors, 4D tensors) voor de diverse lagen (Dense, 1D en 2D Convolution, MaxPool, RNN/GRU/LSTM, Attention, Activation functions, Flatten) en hoe deze met elkaar te combineren zijn.

- A** This is correct - when using a 3x3 kernel with stride 1 and no padding, the output dimensions are reduced by 2 in both height and width ($28-2=26$), while the number of channels becomes equal to the number of filters (32).
- B This would only be the case if padding was applied to maintain the original dimensions.
- C This reduction to 14x14 would occur with stride=2 and padding, not with the given parameters.
- D This is incorrect as convolution layers can handle any number of input channels - the kernels automatically scale with the number of input channels.

9. **les3-rnns-001** — correct: **B**

Learning goals: 3.4: How a simple RNN works (hidden state), 3.5: How GRU and LSTM differ from RNN (gates)

- A While saturating activation functions contribute to the problem, they are not the core reason - even with ReLU the fundamental issue of repeated matrix multiplication remains.
- B** This is correct - the chain rule in backpropagation requires repeated multiplication by the recurrent weight matrix, leading to exponential shrinking or growth of gradients.
- C This is incorrect as it confuses RNN concepts with transformer context windows - RNNs can process sequences of any length through repeated looping.
- D This describes a solution (LSTM's cell state) rather than explaining the mathematical root cause of the problem in simple RNNs.

10. **les3-rnns-002** — correct: **A**

Learning goals: 3.5: How GRU and LSTM differ from RNN (gates), 3.6: What the functions of a gate are (remember, forget, something in between), 3.7: How a gate works (with a hadamard product)

- A** This is correct - if the forget gate's output is zero, the term with h_{t-1} becomes zero, effectively 'forgetting' the previous state.
- B** This is incorrect - this describes the function of the input gate, not what happens when Γ_{u_i} is zero.
- C** This is incorrect - while the contribution from the previous state is zeroed out, the new cell state will still be formed from the input part.
- D** This is incorrect - this is not a halting of the learning process but rather the intended function of the gate.

11. **les3-rnns-003** — correct: **A**

Learning goals: 3.10: How 1D convolutions work with timeseries

- A** This is correct because this is exactly how 1D convolutions work, scanning through time with a sliding kernel.
- B** This is incorrect because 1D convolutions don't process each time step independently with kernel size $\neq 1$.
- C** This is incorrect because 1D convolutions don't require padding with batchsize = 1.
- D** This is incorrect because 1D convolutions don't have memory gates.

12. **les4-hypertuning-001** — correct: **B**

Learning goals: 4.2: wat de curse of dimensionality is (bv met betrekking tot de searchspace), 4.4: Hoe bayesian search en hyperband werken

- A** This is incorrect - random search does not protect against overfitting.
- B** This is correct - Grid Search wastes trials by testing the same values repeatedly while Random Search is more likely to find good combinations within the same budget.
- C** This is incorrect - while randomization helps with exploration, grid search doesn't risk getting stuck in local minima.
- D** This is incorrect - both random and grid search suffer from the curse of dimensionality.

13. **les4-hypertuning-002** — correct: **B**

Learning goals: 4.5: wat een learning rate scheduler is, en hoe je kunt herkennen dat je er een nodig hebt., 4.6: Kent verschillende soorten schedulers (cosine warm up, reduce on plateau) en weet wanneer ze te gebruiken

- A** A warm-up uses a very low learning rate, which slows down initial convergence, it does not speed it up. The goal is stability, not speed.
- B** At the start of training, a network with random weights can produce very large, unpredictable gradients. A high initial learning rate could cause the model's weights to change drastically, destabilizing the training process from the outset. A warm-up starts with a small learning rate and gradually increases it, allowing the model to 'settle in' before learning proceeds more aggressively.
- C** A warm-up involves starting with a low learning rate and increasing it, not starting with a high one.
- D** This describes a learning rate range test or finder, which is a different technique used to estimate a good learning rate, not a warm-up scheduler used during training itself.

14. **les4-hypertuning-004** — correct: **C**

Learning goals: 4.2: wat de curse of dimensionality is (bv met betrekking tot de searchspace), 4.3: wat de voordelen van Ray zijn, 4.4: Hoe bayesian search en hyperband werken, 4.11: De student kan redeneren over de grootte van de hyperparameter ruimte, daar afwegingen in maken (curse of dimensionality) en prioriteiten stellen in de tuning van hyperparameters zoals lossfuncties, learning rate, units, aantal lagen, aantal filters, combinatie van Dense / Convolution.

- A This is incorrect, because you cover only 0.02
- B This is incorrect, because while it is more uniform, it still not enough by a lot.
- C This is correct; you will need to prune inefficient trails, and use the TPE algorithm.
- D This is incorrect; brute force is not an option when covering 0.02

15. **les5-attention-001** — correct: **A**

Learning goals: 5.1: Precision vs Recall trade off, Confusion matrix, ethische problemen van de trade off

- A Correct because the model has recall 20
- B Incorrect because the model has very few false positives (FP=1), so it does not cause unnecessary anxiety; it prioritizes precision.
- C Incorrect because there is a clear and significant trade-off being made.
- D Incorrect as the data (1000 samples) is sufficient to evaluate the model's performance and identify this trade-off.

16. **les5-attention-002** — correct: **B**

Learning goals: 5.2: de motivatie achter attention (vs convolutions), 5.8: kan uitleggen hoe attention behulpzaam is bij timeseries in het algemeen, en NLP in het bijzonder.

- A Incorrect - it identifies attention's strength in long-range dependencies but wrongly suggests it's poor at local patterns.
- B Correct - the core mechanism of attention is to create a pairwise score between every token and every other token, making dependency modeling distance-independent.
- C Incorrect - this describes the mechanism of an RNN instead of attention.
- D Incorrect - this describes the mechanism of a 1D CNN instead of attention.

17. **les5-attention-003** — correct: **A**

Learning goals: 5.2: de motivatie achter attention (vs convolutions), 5.4: Hoe het "reweighing" van word-embeddings werkt met behulp van attention

- A Correct - captures the 'holographic' nature of self-attention where every token's final representation is a function of every other token in the context.
- B Incorrect - describes an RNN and incorrectly states the computational scaling.
- C Incorrect - describes specific efficiency modifications to attention but not the fundamental mechanism of standard self-attention.
- D Incorrect - incorrectly describes the mechanism as local window-based when attention computes scores across the entire context.

18. **les6-autoencoders-001** — correct: **B**

Learning goals: 6.7: Wat de elementen van een siamese network zijn, 6.8: Wanneer je siamese networks zou willen gebruiken

- A This is incorrect as it's a standard classification problem suited for a normal classifier.
- B This is correct as it's a classic 'open-set' learning problem where Siamese networks excel at learning similarity functions for new, unseen examples.
- C This is incorrect as it's a regression task, not a classification or similarity task.

D This is incorrect as it's an unsupervised compression task better suited for an autoencoder.

19. **les6-autoencoders-002** — correct: **B**

Learning goals: 6.1: hoe de architectuur van een autoencoder in elkaar zit, 6.2: Wat een latent space is, en hoe je die kunt visualiseren

A This is incorrect as the loss is calculated against the clean target to force the model to learn to remove noise.

B This is correct as this is the defining characteristic of a denoising autoencoder, forcing it to learn robust features by reconstructing clean data from noisy input.

C This is incorrect as a bottleneck is still crucial and making it too large would allow it to simply learn an identity function.

D This is incorrect as denoising autoencoders are commonly used for images, such as removing grain or watermarks.

20. **les6-autoencoders-003** — correct: **B**

Learning goals: 6.2: Wat een latent space is, en hoe je die kunt visualiseren, 6.5: Hoe anomaly detection met een autoencoder werkt

A While minimizing reconstruction loss is the training objective, this applies only to normal data, not all possible inputs.

B The latent space serves to model the essential characteristics of normal data in a compressed form.

C In autoencoders, the latent space is lower-dimensional than the input space ($d_m < d$).

D This describes clustering approaches to anomaly detection, not the standard autoencoder approach.

21. **les7-graphs-001** — correct: **A**

Learning goals: 7.9: Spectral clustering: Begrijpt dat je de eigenvectors van een Laplacian matrix kunt gebruiken om een graph te clusteren

A For spectral bi-partitioning of a connected graph, the Fiedler vector (the eigenvector of the second-smallest eigenvalue) is used.

B For a connected graph, the smallest eigenvalue is 0, and its corresponding eigenvector is a constant vector (e.g., all ones) which provides no information about how to partition it.

C The largest eigenvalue and its eigenvector relate to other graph properties but are not used for the primary spectral partition.

D While node degree is a useful local metric, spectral clustering is a global method that relies on the structural information captured by the eigenvectors of the Laplacian, not just the local degree.

22. **les7-graphs-003** — correct: **C**

Learning goals: 7.3: Begrijpt wanneer en waarvoor hyperbole embeddings bruikbaar zijn

A Hyperbolic embeddings often require more complex computation than Euclidean embeddings.

B Grid-like structures with equal distances are better represented in Euclidean space.

C Hyperbolic space grows exponentially, making it ideal for hierarchical data with exponential growth patterns like trees, taxonomies, or networks with low-dimensional embeddings.

D The dimensionality depends on the specific application and data; hyperbolic embeddings may use fewer dimensions for certain hierarchical structures but this isn't universally true.

23. **les7-graphs-004** — correct: **A**

Learning goals: 7.7: Begrijpt hoe een graph convolution werkt, en hoe dit te vergelijken is met een 2D convolution

- A** This is the core ‘message passing’ or ‘neighborhood aggregation’ idea of a GCN - a node’s new representation is formed by aggregating information (features) from its one-hop neighborhood, including itself.
- B** This describes a standard CNN - GCNs are needed because graphs do not have a regular grid structure like images, so a fixed 2D filter cannot be applied.
- C** While path lengths are important in graph theory, they are not the basis for the update rule in a standard GCN layer, which focuses on immediate neighbors.
- D** This describes a Graph Attention Network (GAT), which is a different type of GNN - a standard GCN uses a fixed, unweighted (or degree-normalized) averaging, not a learned attention mechanism.

24. **les8-swarm-001** — correct: **B**

Learning goals: 8.5: the students understands what a POMDP is and can explain the differences with an MDP

- A** Both MDPs and POMDPs typically have probabilistic transition functions. Determinism is a specific case, not a defining difference.
- B** This is the fundamental definition. In a POMDP, the state is hidden; the agent only sees observations that are probabilistically related to the state (via the observation function) and must estimate where it is (belief state).
- C** Both models can define state spaces that are either discrete or continuous.
- D** Both MDPs and POMDPs can be solved using dynamic programming (if models are known) or RL (if models are unknown).

25. **les8-swarm-002** — correct: **C**

Learning goals: 8.11: The student understands the principle of differential evolution, PSO and Grey Wolf optimization algorithms, 8.12: the student understands the upsides and downsides of using swarm algorithms for optimization

- A** This is actually an upside. Swarm algorithms are derivative-free and excellent for black-box problems.
- B** They are meta-heuristics and usually do NOT guarantee convergence to the global optimum (unlike convex optimization methods).
- C** Swarm algorithms are stochastic. While they are good at escaping local optima, there is no proof they will always find the exact global solution in finite time, and evaluating a whole population of agents is often slower per iteration than evaluating a single gradient.
- D** Handling multimodal landscapes is one of their strengths; the population helps explore multiple peaks simultaneously.

26. **les8-swarm-005** — correct: **B**

Learning goals: 8.2: The student understands the definition of an MDP ($\mathcal{S}, A, T, R, \gamma$) and is able to apply the concepts to problems, 8.4: The students understands what a policy $\pi^(s) = a$ is*

- A** This confuses the policy with a belief state or a one-step greedy approach; the optimal policy maximizes long-term cumulative reward, not just the next step.
- B** Correct: the optimal policy $\pi^*(s) = a$ maps each state to the action that maximizes expected cumulative discounted reward.
- C** A policy is state-dependent and reactive, not a fixed sequence of actions independent of state.
- D** This describes the transition function $T(s, a, s')$, not the policy.